PhotoFinish: Portable Timerless Probing of Microarchitectural Latencies

Tristan Hornetz, Eduard Ebert, Lorenz Hetterich, Lukas Gerlach, Michael Schwarz
CISPA Helmholtz Center for Information Security

arXiv:2611.09879v1 [cs.CR] 6 Nov 2026

Abstract

Timing measurements are a fundamental building block of microarchitectural attacks, enabling software to convert otherwise invisible latency differences into architectural observations. Thus, restricting high-resolution timers is a common defense, especially in constrained environments such as web browsers. Timerless probing primitives provide an alternative approach. Instead of reading or constructing a timer, they observe architectural events that correlate with the microarchitectural state. However, such primitives typically rely on platform-specific behavior, and none generalizes across the wider computing landscape.

In this paper, we present PhotoFinish, a portable timerless probing primitive that converts latency differences directly into an architectural value. PhotoFinish races an operation under test against a fixed reference in two parallel threads that then write unique values to the same shared location, resulting in a latency-dependent architectural value. PhotoFinish only requires parallel execution and shared memory, without constructing or reading a clock. We evaluate PhotoFinish on 38 CPUs spanning four ISAs and demonstrate reliable cache-state classification across widely different microarchitectures, with comparable fidelity when compiled to WebAssembly and executed inside a browser. We further use PhotoFinish in AES T-table and Evict+Reload attacks, show that it applies beyond caches to TLB and execution-engine latencies, and demonstrate sufficient precision to identify the LLC slice of an address. Our results demonstrate shared-memory races as a generic and precise replacement for explicit timing, showing that restricting timers alone does not prevent software from observing fine-grained microarchitectural latency differences.

1 Introduction

Software-based microarchitectural attacks remain a practical threat to the confidentiality and integrity of modern systems. They exploit effects below the architectural abstraction, such as cache state [45, 72], branch-predictor state [14, 37], TLB state [17, 38], execution-unit contention [5, 53], or DRAM row-buffer state [48]. Transient-execution attacks additionally exploit instructions that execute transiently and leave observable microarchitectural traces [12, 34, 41]. Although these effects are not directly visible in a program's architectural state, they affect execution time. Thus, attackers can infer information across security boundaries by observing small latency differences in otherwise ordinary software execution.

A central building block of many such attacks is a timing measurement primitive. The attacker executes an operation whose latency depends on microarchitectural state and then classifies it as fast or slow. In cache attacks, this operation is commonly a memory access [20, 52, 72]. In other attacks, it may be a branch [4, 14, 37], an instruction sequence contending for a shared execution resource [5, 24], a page-table walk [19, 38], or an instruction's execution time [7]. In all cases, the attacker needs a way to turn a microarchitectural latency difference into an observation that software can use.

Traditionally, attackers obtain this observation using an incrementing timer. The attacker timestamps the operation before and after execution and compares the difference against a fixed threshold. Consequently, a common defense strategy is to restrict access to high-resolution timers: browsers reduce timer resolution and introduce jitter [54, 58, 64], or disable them in specific settings [27, 39, 62]. These restrictions do not remove the underlying latency difference. They only remove the most direct way to observe it. Attackers can compensate by amplifying the targeted difference [6, 49, 57, 61] or constructing software clocks such as counter threads [18, 39, 54].

An alternative is to avoid constructing a timer altogether. Existing timerless primitives expose microarchitectural state through specialized instructions [13, 25, 33, 63, 73, 75] or environment-specific properties of the cache hierarchy [21, 31, 62, 63]. These techniques demonstrate that explicit timestamps are not essential, but they replace the dependence on a timer with assumptions about a particular ISA, processor feature, or cache implementation. This leaves a portability gap: existing timerless primitives do not provide a generic

way to observe latency differences across architectures and restricted execution environments, such as browsers.

In this paper, we ask the following research question:

Can we build a generic primitive that observes microarchitectural latency differences without reading a timer and without relying on specialized architecture-specific features?

To answer this question, we present PhotoFinish, a timerless probing primitive applicable to all major computer architectures and even to restricted environments like web browsers. We base this primitive on an observation by Purnal et al. [49]: A race condition between two parallel threads can reveal microarchitectural timing differences when both threads write to a shared variable after finishing a workload. Specifically, one thread, which we call the *probing thread* `T0`, performs the target operation, whose latency depends on the microarchitectural state. In parallel, a *reference thread* `T1` performs a reference operation with known latency. Finally, both threads write distinct values to the same shared result variable. When both threads complete, the final value of this variable reveals which store serialized last, showing whether the target operation was faster or slower than the reference. The key property of this approach is that the attacker does not need a timestamp. The primitive only requires two threads, shared memory, and sufficient control over instruction ordering.

With PhotoFinish, we develop this concept into a practical, generic, and near-universally applicable probing primitive for microarchitectural latencies. We distinguish between a *padded* and an *unpadded* variant. For sufficiently large latency differences, the unpadded variant works without prior calibration, which is typically required for timer-based primitives. For smaller latency differences, padded PhotoFinish adds a calibrated delay to `T0`. This moves the race boundary in the same way that a threshold moves the decision boundary in a timer-based attack, but without ever reading a clock.

We evaluate PhotoFinish on 38 CPUs spanning x86_64, AArch64, RISC-V, and LoongArch, and 45 platform configurations once hybrid architectures are separated. This breadth allows us to demonstrate PhotoFinish's platform-agnostic properties: For instance, unpadded PhotoFinish reliably separates L1 hits from DRAM accesses on most platforms, and padded PhotoFinish achieves this on all of them. This is despite vastly different core microarchitectures, interconnect topologies, and even instruction sets. A large set of test platforms also enables us to characterize PhotoFinish's performance broadly. We identify the quantity that bounds the resolution of unpadded PhotoFinish– the synchronization latency Δ_{sync} between the two threads – and measure it across our evaluation set. This allows us to estimate the minimum latency difference for which unpadded PhotoFinish works. By quantifying the jitter in Δ_{sync} between runs, we can additionally estimate this for the padded variant.

The results show that PhotoFinish enables timerless variants, e.g., of Flush+Reload [72] and Evict+Reload [39] on most platforms, a conclusion we also verify experimentally. Padded PhotoFinish in particular features a high temporal accuracy, and can even distinguish adjacent cache levels on some target platforms. When multiple measurements of a target latency are permitted, we show that PhotoFinish can reliably resolve timing differences at the level of individual clock cycles. We demonstrate this by recovering an address's last-level-cache (LLC) slice on a 12-slice Intel CPU with 96.9 % accuracy.

PhotoFinish is not limited to native code. We implement PhotoFinish in WebAssembly and evaluate it in Chromium for x86_64, using only the standard `WebWorker` parallelism and `SharedArrayBuffer` memory. We find that the synchronization latency strongly correlates with the native implementation (Pearson $r = 0.930$ across 50 matched configurations), and directly verify that browser-based PhotoFinish can distinguish L1 hits from DRAM accesses on all tested machines. This indicates that PhotoFinish's performance in the browser approaches that of the native implementation, making it a strong primitive for browser-based attacks. Additionally, we show that PhotoFinish remains effective when implemented in Lua, achieving near-perfect classification of cached and uncached accesses despite JIT compilation.

We further compare PhotoFinish with counter-thread timers, a common fallback when explicit timers are unavailable. Like counter threads, PhotoFinish remains affected by thread placement and cache-coherence behavior. Thus, its advantage is not independence from these mechanisms, but that it directly encodes the microarchitectural state into an architectural value instead of constructing a clock from periodically visible counter updates, which requires a (potentially adaptive) threshold. Across physical cores, this direct race usually provides a finer decision boundary and matches or slightly exceeds the classification accuracy of counting threads. Padded PhotoFinish provides the broadest coverage, while SMT-sibling counting threads remain competitive for smaller LLC-to-DRAM latency differences.

Finally, we show that PhotoFinish substitutes for a timer in vastly different scenarios, including complete attacks. We adapt a first-round AES T-table attack by replacing the timed reload step with PhotoFinish. On an Apple M1, where unprivileged code sees only a 24 MHz counter, we construct minimal L2 eviction sets and run an Evict+Reload-based covert channel entirely without a high-resolution timer. These case studies demonstrate that PhotoFinish is a practical primitive in environments where precise timers are restricted. Furthermore, we show the primitive is not specific to caches by detecting TLB residency, AVX wakeup throttling, and subnormal floating-point penalties.

Contributions. Our contributions can be summarized as follows:

- We present PhotoFinish, a generic timerless primitive that converts microarchitectural latency differences into the architectural outcome of a shared-memory race, and implement it on x86_64, AArch64, RISC-V, and LoongArch.

- We evaluate synchronization precision, latency classification, and counter-thread baselines on 38 systems across four ISAs, showing that PhotoFinish outperforms counting threads in many scenarios.
- We implement PhotoFinish in WebAssembly and demonstrate its efficacy inside the browser sandbox.
- We demonstrate that PhotoFinish can replace timed probing in practical settings, including an AES T-table attack and a timerless Evict+Reload-style case study.

2 Background

In this section, we provide the background required for the remainder of the paper. We first introduce software-based cache side channels and the role of memory-access latency (Section 2.1). We then discuss timers and existing timer alternatives (Section 2.2). Finally, we briefly discuss shared-memory and cache-coherence behavior (Section 2.3).

2.1 Cache Attacks

Modern CPUs use caches to compensate for high memory access latencies. When a program accesses a memory address, the access is fast if the corresponding cache line is in a nearby cache, and slower if the line must be fetched from a slower cache level or main memory. Although software does not see this directly, it affects execution time. Cache attacks exploit this timing difference to infer information about another computation [20, 45, 52, 72]. A typical cache attack consists of three phases. First, the attacker prepares the cache state. Second, the victim executes and may access memory depending on secret data. Third, the attacker probes the cache state and infers whether the victim accessed a particular cache line. The probing phase is usually implemented as a timing measurement: the attacker accesses an address and classifies the access as a cache hit or miss based on its latency. Thus, cache attacks observe secret-dependent cache state and translate it into information about the victim.

Flush+Reload. Flush+Reload relies on shared memory between attacker and victim [72]. The attacker flushes a shared cache line from the cache hierarchy, waits for the victim, and reloads the line. A fast reload indicates that the victim accessed the line in the meantime. Flush+Reload is precise because it monitors individual cache lines, but it requires either a flush instruction or another mechanism to evict the target line.

Prime+Probe and Evict+Reload. Prime+Probe does not require shared memory. Instead, the attacker fills a cache set with its own addresses, lets the victim execute, and then measures whether its own addresses were evicted [45]. Evict+Reload combines the reload-based observation of Flush+Reload with eviction sets instead of explicit flush instructions [39]. This makes it applicable on systems where flushing is unavailable or restricted, but it requires finding eviction sets that map to the same cache resources as the target address.

2.2 Timers and Timer Alternatives

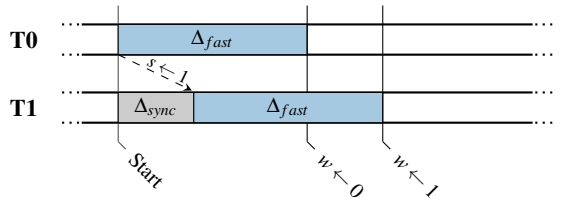
In the context of side-channel attacks, a timer is typically an incrementing counter that lets software compare the duration of two operations. Native attackers can often use architectural timers, performance counters [39], or cycle counters [16], such as `rdtsc` on x86. In constrained environments, such as browsers, precise timers are commonly restricted or coarsened because they enable microarchitectural side channels [54, 64]. Attackers use amplification and repeated measurement to address two distinct limitations of imprecise timers. Amplification increases the latency difference until it exceeds the timer's quantization interval [6, 32, 49, 57]. Statistical aggregation instead reduces random measurement noise when the target permits repeated observations [10, 55]. Averaging cannot, by itself, recover a latency difference that deterministically maps to the same timer value. It is only useful when noise or sampling jitter provides information across observations. Both approaches increase the number of required operations or victim interactions, which is problematic when the interaction is short, rate-limited, or difficult to repeat.

Counting Threads. Counting threads [18, 23, 39, 54] are a common fallback in microarchitectural attacks when hardware or the execution environment lacks a precise timer. One thread continuously increments a shared variable, while the measuring thread reads this variable before and after the operation of interest. The observed counter difference acts as a software clock. Its effective resolution depends not only on the counter loop, but also on how quickly updates become visible to the measuring thread. Consequently, thread placement, cache coherence, scheduler activity, contention, and CPU frequency changes can alter the relationship between counter increments and elapsed time.

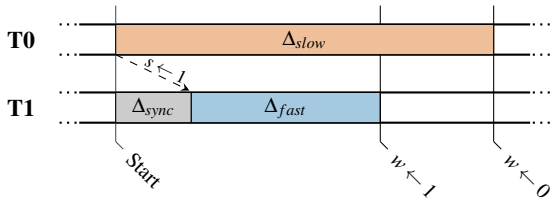
Event-Based Primitives. Some prior attacks avoid conventional timers by observing other events. For instance, weak coherence between data and instruction caches can reveal cache activity on some platforms [21, 31, 62, 63]. On some architectures, this is also possible with specialized instructions such as the Intel TSX extensions [13, 25, 33] or `umwait` [75]. These approaches show that explicit timestamps are not strictly necessary for side-channel attacks, but typically rely on specific ISA features, processor extensions, or execution environments.

2.3 Cache Coherence

Multicore CPUs allow multiple cores to access the same memory location concurrently. To keep memory coherent, CPUs implement a cache-coherence protocol that coordinates cache-line ownership [2, 26, 47]. A core that wants to write to a cache line typically has to obtain exclusive ownership of that line. If the line is not already present in a writable



(a) Fast target: `T1` writes last, yielding $w = 1$.



(b) Slow target: `T0` writes last, yielding $w = 0$.

Figure 1: Unpadded PhotoFinish. `T0` releases `T1` through the shared synchronization word s and immediately executes the target operation. `T1` observes the release after Δ_{sync} and executes the reference operation. In our memory-access instantiation, the target and reference addresses contain distinct marker values, 0 and 1, which are propagated to the final stores. The value remaining in w therefore identifies the path that completed later.

state, the store can trigger coherence traffic, referred to as a read-for-ownership request [28]. Thus, even a cached store can have a variable latency depending on the initial cache state of the written cache line. For a single memory location, coherence establishes an order in which writes become visible. If two threads write different values to the same location, the final value observed in memory corresponds to the write that becomes visible last in this coherence order. This final value is architecturally observable, although the exact write ordering may depend on microarchitectural effects such as cache state, interconnect latency, and core placement. Modern CPUs may also reorder memory operations internally. Out-of-order execution, store buffers, and weak memory models can cause loads and stores to be issued out of program order unless the implementation uses appropriate dependencies or fences [60].

3 The PhotoFinish Primitive

In this section, we introduce PhotoFinish in two variants. We first present the unpadded primitive and derive its correctness condition (Section 3.1). We then introduce a padded variant for smaller latency differences (Section 3.2), discuss the sources of variability in the race (Section 3.3), and describe our implementation and attacker requirements (Sections 3.4 and 3.5).

3.1 High-Level Design

PhotoFinish uses two concurrent threads and a shared result word w. The *probing thread* `T0` executes the target operation, whose latency is either Δ_{fast} or Δ_{slow}. The *reference thread* `T1` executes a fixed reference operation with latency Δ_{ref}. After their respective operations complete, both threads write distinct values to w. Cache coherence serializes these writes, and the value that remains in w identifies the store that became visible last. By making each store data-dependent on the preceding operation, this architectural value reveals which execution path completed later.

Race Construction. Figure 1 illustrates the race. Initially, `T1` spins on a shared synchronization word s. `T0` starts a measurement by writing to s and immediately executing the target operation with latency Δ_x. `T1` observes the write after the synchronization latency Δ_{sync} and then executes its reference operation with Δ_{ref}. Ignoring jitter and assuming marker values '0' and '1', the architectural result is

$$w = \begin{cases} 1, & \Delta_x < \Delta_{sync} + \Delta_{ref}, \\ 0, & \Delta_x > \Delta_{sync} + \Delta_{ref}. \end{cases}$$

In practice, the outcome of measurements close to equality can be influenced by system-noise-induced jitter in Δ_{sync}, as evaluated in Section 4.1.

Correctness Condition. The race distinguishes the fast and slow states whenever the reference path finishes between them, i.e., $\Delta_{fast} < \Delta_{sync} + \Delta_{ref} < \Delta_{slow}$. Unless otherwise stated, we always choose the reference operation to match the known fast case, i.e., $\Delta_{ref} = \Delta_{fast}$, as illustrated in Figure 1. The left inequality holds by construction, and the remaining requirement is $\Delta_{slow} - \Delta_{fast} > \Delta_{sync}$. Hence, Δ_{sync} determines the temporal resolution of unpadded PhotoFinish: smaller synchronization latency permits the primitive to separate smaller latency differences. Inducing a reference delay of Δ_{fast} is often trivial in practice. With memory-access probing, for instance, both the target's fast state and the reference address can be prepared as L1 hits. Unlike a timer-based classifier, unpadded PhotoFinish does not require a platform-specific numerical threshold. The reference operation provides the comparison point directly, and simultaneous execution makes the result a relative ordering rather than an elapsed-time measurement. We show in Section 4 that the latency difference between an L1 hit and a DRAM access exceeds Δ_{sync} on almost all tested platforms. While we focus primarily on memory access timings, PhotoFinish also applies to other microarchitectural latencies (cf. Section 5.4), and even architectural latencies like a function's execution time.

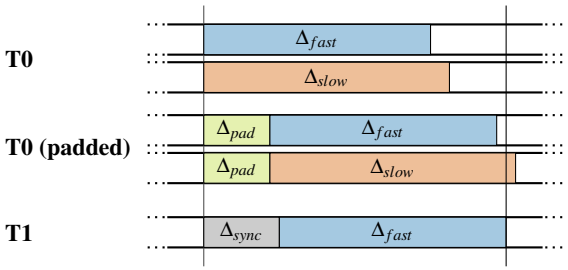


Figure 2: Unpadded and padded PhotoFinish. If the target latency difference is smaller than Δ_{sync}, both target states finish before the reference path. Adding a controlled delay Δ_{pad} to `T0` shifts the race boundary so that only the fast target finishes first.

Takeaway: Unpadded PhotoFinish classifies a latency without a timer or numerical threshold whenever $\Delta_{slow} - \Delta_{fast} > \Delta_{sync}$.

3.2 Padded PhotoFinish

Unpadded PhotoFinish cannot separate two target states when both finish before the reference path. This occurs, for example, when distinguishing adjacent cache levels whose latency difference is smaller than Δ_{sync}. Padded PhotoFinish addresses this limitation by adding a controlled delay Δ_{pad} to `T0` before its final store, as illustrated in Figure 2. A padding value is valid if $\Delta_{pad} + \Delta_{fast} < \Delta_{sync} + \Delta_{ref} < \Delta_{pad} + \Delta_{slow}$. Assuming no jitter, such a value exists for every non-zero latency difference. Thus, padding moves the decision boundary like a numerical threshold in a conventional timer, but without constructing or reading a clock. We calibrate Δ_{pad} using known instances of the fast and slow states. Starting from zero, we increase the padding until `T0` wins the fast case while `T1` wins the slow case. A binary search can locate this interval efficiently: if `T1` wins both cases, the padding is too large; if `T0` wins both, it is too small. The calibration depends on the platform and the target latency pair, but not on an external timing source. We place the padding in `T0` rather than replacing the reference operation with a calibrated delay. A reference path cannot complete earlier than Δ_{sync} after the release. Thus, a delay-only reference cannot distinguish two target latencies that are both shorter than Δ_{sync}. Padding `T0` shifts the effective boundary below Δ_{sync} to cover this case as well. In practice, the minimum distinguishable difference is bounded by jitter in Δ_{sync}, Δ_{fast}, and Δ_{slow}. However, only synchronization jitter is specific to PhotoFinish. Variability in target latencies also affects traditional threshold-based timing classifiers. We quantify the synchronization jitter and the resulting uncertain range in Section 4.1.

Takeaway: Padded PhotoFinish relies on one-time calibration to separate latency differences smaller than Δ_{sync}.

3.3 Controlling Jitter

There are three sources of timing jitter in PhotoFinish: the target and reference operations, the synchronization from `T0` to `T1`, and the coherence arbitration of the final writes to w. The first is inherent to the probed event and also affects timer-based classifiers. The latter two are specific to the race and can be reduced through careful state preparation and instruction ordering.

Memory and Coherence State. We place the synchronization word s and result word w on separate cache lines. With that, releasing `T1` through s does not share a coherence transaction with the final write race on w. Before each measurement, both threads touch the two lines, placing them in their private caches and normalizing the access path. We additionally establish the same clean Shared (`S`) state on both cores, ensuring that either final writer must obtain ownership before transitioning the result line to Modified (`M`). This preparation requires additional care on protocols with owned dirty sharing, such as MOESI [2]. If one core holds a line in Owned (`O`) while the other holds it in Shared (`S`), the owner may upgrade to Modified with less coherence traffic than the sharer, biasing the race. Thus, we flush or evict s and w to the point of coherence before both threads reload them, ensuring that neither core retains ownership. This step is unnecessary for MESI [47] and MESIF [26], where loading the line on both cores already establishes a clean, non-owned shared state.

Instruction Ordering. We implement the latency-critical paths in assembly and connect the relevant operations through true data dependencies. Otherwise, compiler transformations and out-of-order execution can move work across the race boundaries. For memory-access probing, the value loaded from the target or reference address is itself propagated to the final store to w. For more general operations, we carry a marker value through a dependency chain. Note that integrating store operations into such a dependency chain is typically not possible, as they usually do not have an output register. Hence, we implement stores with an exchange instruction if available on the target instruction set.

Thread Placement and System Activity. Δ_{sync} depends on the placement of `T0` and `T1`, including whether they share a physical core, cache hierarchy, interconnect path, or core complex. Hence, scheduler migration would change the decision boundary during an experiment. Our native implementation pins both threads to fixed logical cores, and we evaluate the effect of different placements explicitly in Section 4.1.

3.4 Implementation

Figure 3 shows the implementation at a high level. The reference thread continuously prepares the reference operand and the shared state while waiting for a request from `T0`. A short pre-synchronization handshake then moves `T1` into a tight spin loop on s, reducing variation in its response path.

T0: Probing Thread

```
prepare_shared(s, w)

# pre-synchronize
pre_s = 1
await pre_s == 0

# execute target
s = 1

r = target_op()    # marker 0
w = r

```

T1: Reference Thread

```
prepare_shared(s, w)
while (pre_s == 0) {
  prime(reference)
  access(s)
  access(w)
}
# synchronize
pre_s = 0
await s == 1
r = reference_op() # marker 1
w = r

```

Figure 3: High-level pseudocode for unpadded PhotoFinish. `prepare_shared` normalizes the cache and coherence state of s and w; `target_op` and `reference_op` propagate distinct marker values through data-dependent result stores. We omit the reset logic between measurements for clarity.

After `T0` writes to s, both threads execute their respective operations and propagate distinct marker values to w. A separate completion flag (omitted here) ensures that `T0` reads w only after `T1` has issued its result store. For native experiments, we implement both threads entirely in assembly and expose bindings to higher-level languages. We provide implementations for x86_64, AArch64, RISC-V64, and LoongArch64. The implementations differ primarily in assembly syntax and memory-ordering details. The primitive itself requires only loads and stores, concurrent execution, and shared memory. For padded PhotoFinish, we generate Δ_{pad} with a chain of dependent register-only additions whose final value feeds the result store. Varying the chain length provides fine-grained control over the padding without relying on a timer.

3.5 Threat Model

We assume an attacker with unprivileged code execution on the target machine. PhotoFinish inherently requires (i) two threads that can execute concurrently, (ii) writable shared memory for synchronization, completion, and the result word, and (iii) the ability to invoke the target and reference operations. It does not require a timestamp source, performance counter, privileged instruction, or ISA-specific extension. Our evaluated native implementation also assumes the attacker can keep the two threads on stable logical cores, e.g., through the affinity interface available to unprivileged users on Linux and Windows. Without explicit affinity control, scheduler migration increases Δ_{sync} variability and may require repeated measurements, but this does not change the construction. Our padded implementation generates dependency chains at runtime for convenience. However, runtime code generation is not necessary: pre-generated chains or a dependency-preserving loop can provide the same controlled delay.

4 Evaluation

We evaluate four properties of PhotoFinish: the synchronization precision that bounds its temporal resolution (Section 4.1), its single-shot accuracy when classifying cache states (Section 4.2), its performance relative to counting-thread timers (Section 4.3), and its applicability inside a web browser (Section 4.4). Our test set contains 38 physical CPUs spanning x86_64, AArch64, RISC-V64, and LoongArch64. Seven of these CPUs contain heterogeneous core types. Treating each core type as a separate platform yields 45 native platform configurations. Tables 1 to 3 in Appendix A report full per-platform results. Unless stated otherwise, we pin `T0` to logical core 0 and `T1` to the next physical core. On the 20 configurations supporting simultaneous multithreading (SMT), we repeat each experiment with the threads on sibling logical cores. Where possible, we disable hardware prefetchers. We express short delays in *add-cycles*, defined as the latency of one register-only `add` in a dependency chain. This unit is portable across our four ISAs and avoids conflating core cycles with fixed-frequency timestamp-counter ticks. On many microarchitectures, one `add`-cycle corresponds to one core cycle [3, 28].

4.1 Synchronization Precision

The resolution of unpadded PhotoFinish depends on the synchronization latency Δ_{sync} and, more importantly for padded PhotoFinish, on its run-to-run jitter. Thus, we measure both with respect to thread placement.

Method. To measure Δ_{sync}, we instantiate a padded race without target or reference memory accesses. After `T0` releases `T1` through the synchronization word, `T0` first executes a dependency chain of d register-only `add` instructions, while `T1` writes after observing the release. Thus, the result compares the controlled padding d against Δ_{sync}. We execute 2048 such races for each $d \in [0, 1000)$ and record the probability $p(d)$ that `T0` wins. At $d = 0$, `T0` normally wins, and as d increases, the winner transitions to `T1`. On most CPUs, this transition zone consists of multiple d values where both threads have a high chance of winning. This is illustrated in Figure 4. We call this the *uncertain range* U, and define the lower endpoint as the largest d for which $p(d) \geq 95\,\%$, and its upper endpoint as the smallest d for which $p(d) \leq 5\,\%$, i.e., U brackets the transition. Its midpoint estimates Δ_{sync}, while its width captures the observable synchronization jitter. The upper edge of U provides a conservative bound for unpadded PhotoFinish, whose condition is $\Delta_{slow} - \Delta_{fast} > \Delta_{sync}$. For padded PhotoFinish, the width $|U|$ approximates the minimum latency separation that can be classified reliably with one race. Smaller differences can still be recovered by combining repeated measurements, as we demonstrate in Section 5.3. We run this experiment on all 45 platform configurations. To study topology effects, we keep `T0` on core 0 and place `T1`

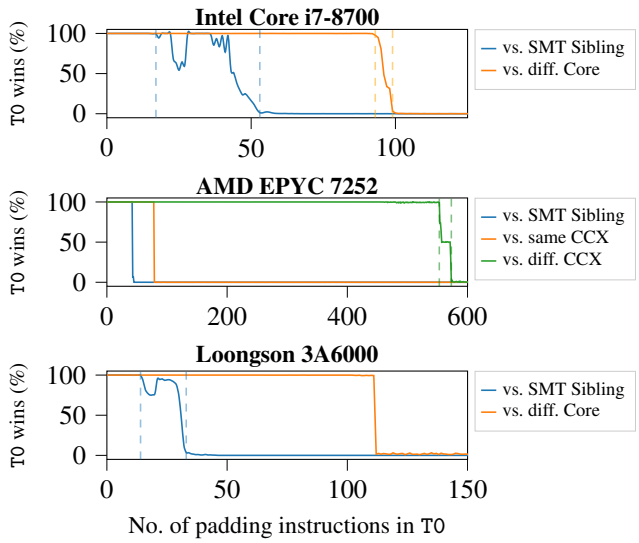


Figure 4: Synchronization transition on representative CPUs ($n = 2048$ races per padding value). Increasing the dependent padding in T0 moves the winner from T0 to T1. The transition midpoint estimates Δ_{sync}. The shaded uncertain range captures values for which neither thread wins at least 95 % of races. Core placement affects both quantities.

on every other logical core. All reported values of Δ_{sync} are midpoints of the corresponding uncertain ranges.

Cross-Core versus SMT Placement. SMT siblings provide a shorter synchronization path because both threads share the private cache hierarchy. However, they also share frontend and execution resources, and we observe a substantially wider uncertain range on most SMT configurations. Thus, SMT reduces the mean synchronization latency but generally worsens single-shot temporal precision. For the remainder of the paper, we prefer separate physical cores unless stated otherwise. Across separate physical cores, the exact core pair has little effect on many CPUs. The measured Δ_{sync} is identical across placements on 8 platforms and varies by fewer than 10 add-cycles on another 21. The main exception is topology that crosses an internal interconnect boundary. For example, on the AMD EPYC 7252, Δ_{sync} is 78 add-cycles when both threads reside in the same core complex (CCX), but averages 563 add-cycles across CCXs. We observe the same behavior on all tested multi-CCX AMD systems: selecting cores within one CCX significantly improves the race boundary. On several platforms, U is only one add-cycle wide, the finest granularity our setup resolves. Repeated experiments on these systems distinguish chains differing by one dependent add. While this result characterizes the synchronization path, variability in the target operation can widen the effective transition during an attack. Accordingly, the values in Appendix A are optimistic lower bounds on the total classification noise.

Predicting L1-versus-DRAM Feasibility. Knowledge of Δ_{sync} enables predicting the success of, e.g., a cache-state classifier. To demonstrate this, we sweep Δ_{pad} from 0 to 1000 add-cycles for both an L1 hit and a DRAM access, taking 2048 samples per case and padding value. We conservatively estimate the difference between an L1 hit and a DRAM access Δ^{L1}_{DRAM} as the separation between the two transition regions: the highest padding for which the L1 case still wins more than 5 % of races and the lowest padding for which the DRAM case wins fewer than 95 %. When running on core 0 and a separate physical core, Δ^{L1}_{DRAM} exceeds Δ_{sync} on all but two platforms, Core i5-6400T and Ampere Altra Q64-30. The unpadded condition therefore predicts successful L1-versus-DRAM classification on the remaining platforms. In contrast, Δ^{L1}_{DRAM} exceeds $|U|$ on every platform, predicting that calibrated padding can separate the two states throughout the test set. The direct classification experiments below, together with the AES case study in Section 5.1, confirm the padded prediction on every platform. Both platforms predicted to fail without padding indeed do, although two further platforms fall short for reasons we discuss below.

4.2 Cache-Latency Classification

In this section, we evaluate PhotoFinish as a primitive for inferring a cache line's state. We consider L1 hits versus DRAM accesses, common in Flush+Reload-style attacks [72], and LLC hits versus DRAM accesses, as required by LLC Prime+Probe [42]. For padded PhotoFinish, we additionally test the smaller L1-versus-L3 and L1-versus-L2 differences.

Method. For every combination of cache state for the address and padding value, we collect 2^{14} samples from each class, with Δ_{pad} from 0 to 500 add-cycles. Note that padding zero is the unpadded construction. We report the F_1 score with the fast state as the positive class. We prepare cache levels using physical-address-based eviction sets, as detailed in Appendix B. Where a sufficiently precise timer is available, we independently verify that eviction leaves the target at the intended cache level. We cannot perform this validation on the RISC-V and LoongArch64 systems or on Huawei Kunpeng 920. Thus, their L2/L3 results are conservative: a low score may indicate imperfect state preparation rather than insufficient resolution of PhotoFinish. For LLC-versus-DRAM measurements on SMT siblings, T1 also accesses an LLC-resident reference line. With separate physical cores, we instead retain an L1-resident reference because Δ_{sync} already shifts the reference path beyond typical LLC latency. Avoiding an LLC access in T1 removes interconnect-induced jitter [46] and improves the decision boundary.

Unpadded PhotoFinish. For L1 versus DRAM on separate physical cores, 41 of 45 platforms achieve $F_1 \geq 0.97$. The four exceptions expose two distinct failure modes. On Huawei Kunpeng 920 and the efficiency cores of Core Ultra 5 325, DRAM latency varies enough that some nominally slow

probes complete before the reference path. On Core i5-6400T and Ampere Altra Q64-30, the L1-to-DRAM separation is too small relative to Δ_{sync}, violating the unpadded condition. Among all configurations, 38 produce fewer than 20 errors across 32 768 classified accesses, and 4 classify every access correctly. All tested RISC-V and LoongArch64 platforms achieve high scores, showing that the unpadded primitive supports timer-free, calibration-free Flush+Reload-style attacks beyond x86 and ARM. On SMT siblings, all 20 platforms exceed $F_1 = 0.99$ for L1 versus DRAM, with 18 platforms having fewer than 20 errors and 8 platforms being perfect. The shorter synchronization path dominates the additional SMT jitter for this large latency difference. For LLC versus DRAM, the cross-core configuration again performs strongly: 37 of 45 platforms exceed $F_1 = 0.99$, including 5 perfect classifiers. These platforms support an unpadded, calibration-free LLC Prime+Probe readout. On SMT siblings, 6 of 20 platforms exceed $F_1 = 0.99$. Here, the smaller cache-latency separation is more sensitive to the wider uncertain range, confirming the placement result from Section 4.1.

Padded PhotoFinish. Padding converts a single race boundary into a tunable classifier. Figure 5 illustrates how padding allows tuning the classifier on a sample Xeon Gold 6346. At zero padding, both the fast and slow target may finish before `T1`, resulting in an unusable classifier with $F_1 \approx 2/3$. As padding increases, the slow state crosses the reference boundary first and the fast state later. Between these transitions, both classes are classified accurately. We define the *golden range* G as a contiguous interval of padding values for which $F_1 > 0.98$. Its width measures calibration tolerance: a wider G makes the classifier less sensitive to small shifts in execution conditions. Every platform has a non-empty golden range for L1 versus DRAM. Thus, padded PhotoFinish provides timer-free Flush+Reload-style classification on all tested CPUs, including platforms where the unpadded condition fails. For LLC versus DRAM, 42 of 45 platforms have a golden range. The finer L1-versus-L3 difference is separable with one race on 32 platforms, and even L1 versus L2 is separable on 15. The latter includes the efficiency cores of Core i9-12900H and the Apple M1. On the M1, the achieved granularity significantly exceeds that of the unprivileged architectural timer [8, 39]. With SMT placement, all 20 platforms retain a golden range for L1 versus DRAM. Smaller differences are less robust: 13 platforms expose a golden range for L1 versus L3, and 7 for LLC versus DRAM. This confirms the tradeoff established in Section 4.1, i.e., SMT minimizes Δ_{sync}, whereas separate physical cores usually minimize the transition width.

Takeaway: Unpadded PhotoFinish provides calibration-free L1-versus-DRAM classification on 41 of 45 cross-core platforms. Calibrated padding extends coverage to all tested CPUs and can even classify adjacent cache levels.

4.3 Comparison with Counting Threads

Counting threads are the standard timing fallback when an explicit high-resolution timer is unavailable [18,23,39,54,56,70]. They require the same basic resources as PhotoFinish, i.e., parallel execution and shared memory, but expose time through periodically visible counter updates. A cache attack must then calibrate a numerical threshold over this emulated clock. We compare the two approaches first at the level of temporal resolution and then through direct cache-state classification.

Counting-Thread Implementation. Our assembly implementation is a loop that increments a register and stores it to shared memory, requiring three instructions per iteration. We evaluate all 45 native platform configurations cross-core and the 20 SMT configurations on sibling threads.

Effective Counter Resolution. The relevant resolution is not the local loop throughput but the interval between counter values observable by another core. Store buffering, cache ownership, and coherence propagation can combine or delay updates. We call the mean interval between globally observed changes the *edge width*. A branch-free observer repeatedly loads the counter and counts value changes. We measure the wall-clock time required to observe 10 000 000 changes using a coarse OS timer, then convert it to `add`-cycles by benchmarking 65 536 repetitions of a 1024-instruction dependent-add chain. The observer introduces a small positive bias, but the counter loop itself takes fewer than 7 `add`-cycles per iteration on every platform, whereas the measured edge width is substantially larger on nearly all of them. The cross-core edge width is higher than Δ_{sync} by more than 30 `add`-cycles on 28 of 45 platforms, while the opposite occurs on only 5. Thus, the unpadded race usually has a finer boundary than a cross-core counting thread. On SMT siblings, edge width and Δ_{sync} are typically within 15 `add`-cycles, although Loongson 3A6000 and Ryzen 9 9950X are outliers at 3 and 249 `add`-cycles, respectively. For padded PhotoFinish, the relevant quantity is $|U|$ rather than Δ_{sync}. On 19 of 20 platforms, the cross-core uncertain range is narrower than the edge width of an SMT-sibling counting thread, indicating a substantially finer calibrated decision boundary.

Direct Classification. We use the counting thread as a timer for the same L1-versus-DRAM and LLC-versus-DRAM classifications. For each pair, we profile the mean fast and slow readings, set the threshold to $T_{fast} + \frac{1}{2}(T_{slow} - T_{fast})$, and classify the fast and slow cases over 65 536 iterations. In the cross-core setting, counting threads exceed $F_1 = 0.97$ on 36 of 40 platforms for L1 versus DRAM and 32 for LLC versus DRAM. On the same subset, unpadded PhotoFinish reaches 37 and 33, respectively, despite requiring no numerical threshold. Padded PhotoFinish reaches all 40 platforms for L1 versus DRAM and 35 for LLC versus DRAM. Thus, cross-core PhotoFinish matches or slightly exceeds the counting-thread baseline, and padding further increases coverage. The SMT result is more nuanced. Counting threads exceed $F_1 = 0.97$ on

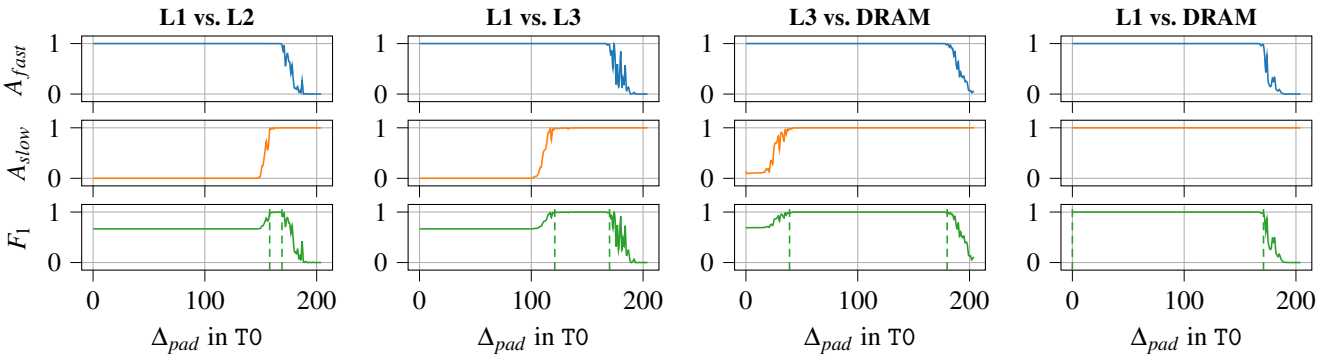


Figure 5: Cache-state classification on Xeon Gold 6346 with cross-core placement ($n = 16384$ per class and padding value). The fast-state accuracy A_{fast} decreases at a larger padding value than the slow-state accuracy A_{slow}. Their overlap forms the golden range G, in which the combined classifier achieves $F_1 > 0.98$.

16 of 18 platforms for both latency pairs. Unpadded PhotoFinish reaches all 18 for L1 versus DRAM but only 7 for LLC versus DRAM. Padded PhotoFinish exceeds $F_1 = 0.98$ on all 18 and 9, respectively. Consequently, an SMT-sibling counting thread remains the stronger baseline for the smaller LLC-to-DRAM difference, but at the cost of threshold calibration.

Takeaway: Across physical cores, PhotoFinish is at least as accurate as counting-thread timing while unpadded PhotoFinish eliminates threshold calibration. Padded PhotoFinish provides the broadest coverage.

4.4 Browser Execution

We demonstrate that PhotoFinish in WebAssembly is a practical alternative to counting threads in browsers. Browsers are an important restricted environment for microarchitectural attacks because they execute untrusted code, support parallel execution through Web Workers, and expose shared memory through `SharedArrayBuffer` under cross-origin isolation [44, 61, 65, 68].

Implementation and Scope. We implement both threads in WebAssembly and execute them in Chromium v147.0 on x86_64. Using JavaScript, we rewrite the WebAssembly bytecode to insert a variable-length chain of dependent additions for Δ_{pad}. Inspection of the generated machine code confirms that Chromium emits the intended register-only dependency chain. The primitive itself uses only Web Workers and `SharedArrayBuffer`. Standard browser code cannot execute `clflush`, and this experiment requires known cache states to measure classification accuracy. Thus, we use a patched Chromium build to prepare and label the probe line through native cache-management instructions. We do not use these hooks for synchronization, padding, or readout, and the WebAssembly primitive does not normalize the coherence state of s and w as the native implementation does. We repeat the synchronization and L1-versus-DRAM experiments with the same cross-core and SMT placements as in native code, again treating heterogeneous core types separately. Per padding value, we collect 2048 synchronization races and 4000 cache-state samples.

Results. The WebAssembly and native results are strongly correlated (Pearson $r = 0.930$ across the matched configurations). Browser execution increases the mean Δ_{sync} from 98.7 to 129 `add`-cycles and widens the mean uncertain range from 23.7 to 37.8 `add`-cycles. The additional latency and jitter are consistent with the longer generated response path and the missing coherence-state preparation for s and w. Despite this reduced resolution, unpadded WebAssembly PhotoFinish obtains $F_1 \geq 0.99$ in 46 of 50 configurations for L1 versus DRAM. Padded PhotoFinish yields a golden range on all 50 configurations. Its mean width is 84.8 `add`-cycles, close to the native mean of 86.9, and the per-configuration widths remain strongly correlated (Pearson $r = 0.853$). Thus, the shared-memory race survives browser compilation with only a moderate increase in synchronization noise.

Takeaway: WebAssembly preserves the behavior and accuracy of PhotoFinish: padded classification succeeds in every tested browser configuration.

5 Case Studies

In this section, we show that PhotoFinish can substitute for a timer in 5 case studies. Section 5.1 demonstrates PhotoFinish in a timerless attack against AES. In Section 5.2, we show that PhotoFinish can find minimal eviction sets, and use them for an Evict+Reload-based covert channel on an Apple M1 CPU. Section 5.3 demonstrates that PhotoFinish can detect the LLC slice of an address with aggregated measurements. Section 5.4 shows that PhotoFinish also resolves non-cache latencies. Finally, in Section 5.5, we show that PhotoFinish does not strictly require access to binary code by implementing the primitive in Lua.

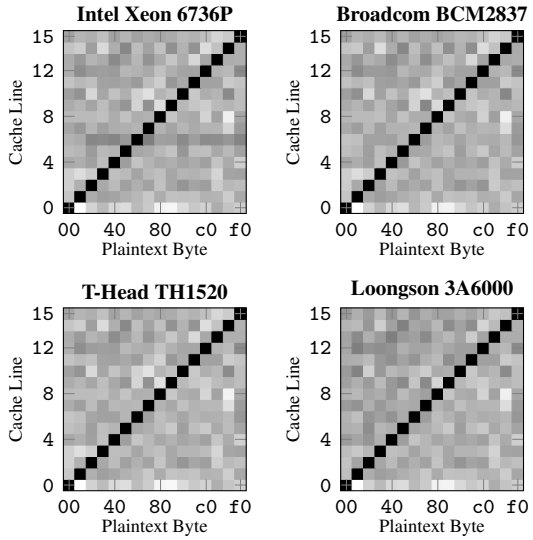


Figure 6: A selection of heatmaps produced by the T-table attack using unpadded PhotoFinish on 4 different CPUs. All show the diagonal line expected when the first key byte is 0.

5.1 AES T-Table Attack

To evaluate PhotoFinish's performance in a real-world attack scenario, we use it in a Flush+Reload-style attack targeting OpenSSL's T-table-based AES implementation. Specifically, we mount a first-round chosen plaintext attack to partially recover the first key byte from memory access patterns [45]. This attack has emerged as a standard benchmark for cache attacks, making our results directly comparable to a large body of existing work [11, 13, 15, 16, 20, 40, 49, 50, 63, 66, 69].

Setup. We specifically target OpenSSL v.3.5.0, compiled for all 4 architectures in our evaluation set using GCC v15.3. OpenSSL's T-table AES implementation uses no architecture-specific code, enabling us to mount the experiment on all 38 CPUs in our evaluation set. However, we exclude Apple's M1 and M2 architectures, as they feature a cache line size of 128 B at the point of coherence, whereas the attack requires 64 B. We run unpadded PhotoFinish on separate physical cores, with `T0` on core 0 and `T1` on the physical core with the next-highest ID. As with the experiments in Section 4, we treat CPUs with hybrid architectures as two test platforms. With 5 of our 36 remaining CPUs being hybrids, this results in 41 total platforms. Since the memory containing the T-tables is physically shared between the attacker and victim, we can use PhotoFinish in a Flush+Reload-like attack. We flush each relevant cache line to DRAM, perform the encryption, and then probe the cache lines using PhotoFinish. This corresponds to the L1 vs. DRAM configuration tested in Section 4.2. On platforms without a flush instruction, we evict cache lines using the method described in Appendix B. In each iteration of the attack, we twice encrypt a chosen 128-bit plaintext block with 0 as the first key byte. We set the first byte of this block to a multiple of 16, with the rest containing random data generated from a fixed seed. Overall, we run 1000 probing iterations for each possible setting of the first byte, probing all 16 relevant cache lines in each iteration. We analyze the resulting access patterns to determine whether the attack is successful and repeat the attack 1000 times per platform to gauge its reliability.

Results. Of the 41 tested platforms, 37 have a success rate of 100 %. One platform, the Broadcom BCM2712, reaches 99.9 %. Figure 6 shows a representative sample of the heatmaps the attack yields. All clearly show the diagonal line expected when the first key byte is 0. Moreover, the noise patterns in non-diagonal cells show similarities across machines, indicating high probing accuracy. On average across all platforms, the median runtime is 315 ms. The lowest average runtime is 218 ms on the AMD Ryzen 9 7950X3D, and the highest is 17.4 s on the Microchip MPFS025T. There are only 3 machines on which the attack is unreliable: the StarFive JH7100 (3.5 % success rate), the Ampere Altra Q64-30 (55 % success rate), and the Intel Core i5-6400T (0 % success rate). For the latter two, this is expected, as Section 4.2 predicts: on these platforms Δ^{L1}_{DRAM} does not exceed Δ_{sync}, so the precondition for the unpadded variant does not hold. We confirm that a padding of 75 `add`-cycles restores a 100 % success rate. For the StarFive JH7100, which has only two parallel threads, we believe the low success rate results from its small cache (2 MB LLC) combined with its high susceptibility to outside interference due to its low core count.

5.2 Evict+Reload

In addition to demonstrating PhotoFinish in a Flush+Reload attack, we show that attackers can use it to find eviction sets without a precise timer. For this, we implement a cross-core covert channel on an Apple M1 system using Evict+Reload on the L2 cache. On this architecture, traditional timing-based cache attacks are challenging since only a coarse-grained 24 MHz timer (CNTVCT) is available.

Finding Minimal Eviction Sets. The M1 processor uses a complex hash function to index its L2 cache [73]. Its hash depends on physical address bits outside the page offset, so we cannot compute minimal eviction sets from virtual addresses alone. Instead, we start with an excessive eviction set of 1536 addresses that have the same page offset as the victim address, i.e., a 24 MB array. We then remove one address at a time and use the padded PhotoFinish primitive to detect whether the eviction set can still evict the target address from the L2 cache. If eviction still succeeds, we remove this address from the eviction set, and repeat until we reach 12 addresses, i.e., the L2 cache's associativity. While we only use minimal eviction sets in our experiment, note that minimality is not a strict requirement. The initial set of 1536 could also be used with a potential reduction in throughput.

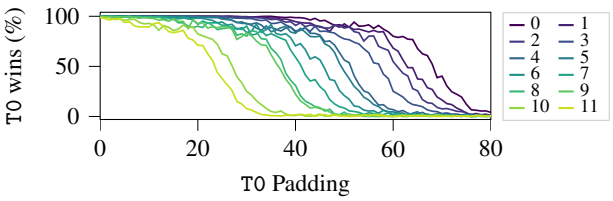


Figure 7: Win rate of `T0` with the probe address in different LLC slices of an Intel Core i9-13900K ($n = 4096$ per sample). The threads run on cores 0 and 1, which are SMT siblings.

Synchronization. We use the coarse-grained `CNTVCT` timer to lay out fixed time slots for each transmitted bit. To signal the start of a transmission, the sender holds the one-signal for eight consecutive time slots, which the receiver decodes as a start byte of `0xff`. The receiver discards all bytes until it observes this value. If no coarse-grained timer is available, an additional cache line could transmit a clock signal, or post-processing could align the measurement to bits.

Receiver. The receiver uses two threads with the PhotoFinish primitive in a loop. Since Δ_{sync} is sufficiently long, we use an L1 cache hit and additional padding instructions instead of an L2 cache hit in the reference thread. After each measurement, we evict the shared cache line from the L2 cache using a minimal eviction set. A counter is initialized to 0 at the start of a time slot. On each cache hit, it is incremented; on a cache miss, it is decremented. If the counter is positive at the end of a time slot, the received value is 1; otherwise, it is 0.

Sender. To send a 1, the sender repeatedly accesses the shared cache line in a loop. To send a 0, the sender evicts the shared cache line from its L1 cache. This is necessary since eviction by the receiver from the L2 cache can fail if the value is still present in the L1 cache of a different core.

Results. We run the covert channel on the M1's Firestorm performance cores at 30 kB/s and transmit 64 kB of data, conducting 200 runs in total. We find a minimal eviction set in 163 of these runs (81.50 %) with a median construction runtime of 289 ms. If an eviction set is found, a median of 96.87 % of bits is received correctly without error correction. This shows that the PhotoFinish primitive can reliably conduct eviction-based cache side-channel attacks on platforms without highly accurate timers.

5.3 Determining LLC Slices

The LLCs of multicore Intel CPUs are segmented into *slices*, each typically grouped with one physical CPU core [15, 30, 43, 51]. Slices are interconnected in a ring- or mesh-like routing structure, and LLC access latency depends on how far a slice is from the core issuing the operation. A complex microarchitectural hash function over the physical address bits determines which slice an address uses. We show that with multiple aggregated measurements, PhotoFinish can determine an address's LLC slice. For this experiment, we use the Intel Core i9-13900K, which features 12 LLC slices on a bidirectional ring interconnect. The hash function on this CPU is known [51], giving us reliable ground truth. To reliably evict addresses to the LLC, we use the eviction strategy outlined in Appendix B. We first measure `T0`'s win rate at different padding levels, with the probe address in different LLC slices and the threads on SMT siblings. The results of this experiment are shown in Figure 7. We observe that the point at which the win rate drops below 50 % is different for each LLC slice, with the relative differences varying from 45 `add`-cycles for slices 0 and 11 to just 1 `add`-cycle for slices 8 and 9. However, we also observe that no padding level creates a scenario where, e.g., an address from slice 8 always causes `T0` to win and an address from slice 9 always causes it to lose. To determine the cache slice, we therefore use the location of the win rate's 50 % threshold as a stable marker. Hence, we must take multiple measurements per test address with different padding levels. We measure PhotoFinish's performance for determining an address's cache slice in a separate experiment. It consists of two steps: A *calibration phase*, which determines the 50 % thresholds of the previous experiment using addresses with known cache slices, and an *evaluation phase*, which determines the cache slice of 1000 random addresses and compares it to the ground truth. In the calibration phase, we determine the 50 % thresholds with the threads pinned to SMT sibling cores 0 and 1 and cores 14 and 15, giving us measurements from two points on the ring. The evaluation phase mirrors this, determining the 50 % threshold of a test address in both configurations, using 512 measurements per padding level. To determine the slice, we treat the calibration thresholds and the two test-address thresholds as points in a two-dimensional space, and choose the slice whose calibration point is closest to the observed test-address thresholds. Using this technique, we observe a slice detection accuracy of 96.9 % with 1000 random addresses. This means that PhotoFinish can distinguish differences on the order of individual cycles in scenarios where multiple measurements are permitted. The uncertain range for SMT siblings of this CPU in Section 4.1 is 31 `add`-cycles wide, confirming that multiple measurements can further improve the temporal resolution.

5.4 Beyond Cache-Access Latencies

While this paper focuses mostly on cache access latencies, PhotoFinish can, in principle, distinguish any two microarchitectural timing differences. We also show it can profile other stateful components, such as the TLB and parts of the execution engine.

AVX Wakeup Throttling. On many Intel CPUs, 256-bit AVX instructions executed after a prolonged period of non-AVX usage are subject to a transient performance penalty. In this "cold" state, latencies of AVX instructions are significantly higher [55]. Haj-Yahya et al. show that this is due to

various current management mechanisms that throttle SIMD instruction throughput while the processor adjusts voltage and frequency settings [22]. Padded PhotoFinish can detect this penalty in AVX-512 instructions. For this experiment, we replace the memory access to the probe address in `T0` with an AVX-512 fused multiply-add instruction. Specifically, we add a single `vfnmsub231pd` instruction with two `vmov` instructions to transfer input and output between SIMD and GP registers. We also remove the reference memory access in `T1`, relying entirely on the padding in `T0` to separate the two cases. For the fast case, we zero all AVX registers and then pre-warm the AVX-512 units by spinning on a series of AVX-512 instructions for 5 µs before measurement. For the slow case, we spin on `rdtsc` for 1 ms, not executing any AVX instructions. We run this experiment on an Intel Xeon 6736P with the threads pinned to cores 0 and 1 (i.e., separate physical cores), taking 512 measurement samples per case and padding level. Starting at a padding level of 159 `add`-cycles, we observe a golden range of 222 `add`-cycles in which the combined F_1 score is above 0.98. In all but 5 outlier cases in this range, the F_1 score is > 0.995. This shows that PhotoFinish can reliably detect this timing difference.

FP Operations with Subnormal Operands. Floating-point standards such as IEEE-754 [1] define a special number class known as subnormal (or sometimes called denormal) numbers. In the binary interchange formats, a subnormal is encoded with an all-zero biased exponent field and a non-zero trailing significand, allowing the representation of values very close to zero. This fills an otherwise unused part of the encoding space but adds complexity when implementing IEEE-754-compliant floating-point arithmetic in hardware. Operations involving subnormal numbers often incur a performance penalty [7, 36], with Intel CPUs typically relying on microcode assists for their implementation [29]. To measure this with PhotoFinish, we replace the memory access to the probe address in `T0` with an AVX2 floating-point division, and let it operate on normal numbers for the fast case and subnormal numbers for the slow case. As in the previous experiment, we remove the reference memory access in `T1`. We run this experiment on SMT sibling cores of an Intel Xeon 6736P, taking 4096 samples per configuration and padding level. We observe an F_1 score of 0.9991 at padding level 0, with the golden range extending to 31 `add`-cycles. Hence, this timing difference can be reliably detected even with unpadded PhotoFinish.

TLB Residency. PhotoFinish can detect whether a CPU's TLB caches a translation for a given virtual address. We verify this with an experiment on an Intel Core i9-13900K. In the fast case, we access the probe address before measurement. In the slow case, we clear it from the TLB using the `invlpg` instruction and access a virtual alias of the probe address's physical location. This ensures that any timing differences originate only from the TLB, with the address in the L1d cache in both cases. For each case, we take 16 384 samples per padding level with the threads pinned to separate physical cores. We observe a golden range of 39 `add`-cycles, starting at padding level 139, meaning the padded PhotoFinish can reliably detect the timing difference. If we additionally flush the relevant page table entries to DRAM with `clflush` before probing, we observe perfect accuracy even without padding, and a golden range of 178 `add`-cycles.

5.5 PhotoFinish in Lua

While previous experiments rely on direct access to binary code or simple environments like WebAssembly to implement PhotoFinish, the low-level capabilities these provide are not strictly required. To demonstrate that PhotoFinish remains effective in a higher-level language, we devise an experiment with unpadded PhotoFinish for the LuaJIT environment (version 2.1.1703358377). It follows the same structure as the WebAssembly implementation in Section 4.4, with the primitive implemented entirely in Lua and a simple C library to prepare the target address's cache state. Additionally, we use LuaJIT's foreign function interface to create a shared memory region. As with WebAssembly, we omit cache state preparation for w and s. We evaluate this experiment in the L1 vs. DRAM configuration, running on an AMD EPYC 7252 with the threads on separate physical cores. Overall, we take 1 000 000 samples each for the cached and the uncached case. We observe just 120 misclassifications in the cached case and 150 in the uncached case, resulting in a combined F_1 score of > 0.999. This shows that PhotoFinish is feasible in higher-level languages.

6 Discussion

In this section, we discuss PhotoFinish's limitations, potential mitigations, and related work.

6.1 Limitations

No Direct Timing Measurement. Compared to a precise hardware timer, PhotoFinish's primary limitation is that it does not measure timing directly. Instead, it compares two timings and reveals which is faster. In cache attacks, this is typically not an issue. Attacks like Flush+Reload or Prime+Probe reveal only whether an address was accessed, so absolute timing values are irrelevant. However, this complicates attacks with more than two states to distinguish (cf. Section 5.3, where there are 12). In this scenario, attacks using PhotoFinish need multiple measurements, whereas a precise timer can, in theory, distinguish them with a single sample.

Precisely Controlled Padding. The padded variant requires a mechanism for selecting a precisely controlled delay sequence. Our native and WebAssembly implementations generate the required dependency chain at runtime. Many higher-level languages do not expose comparable control over generated code. Such environments can instead use loop-based

padding or ship a finite collection of precompiled padding functions. Both approaches may introduce additional jitter or provide coarser calibration steps.

6.2 Related Work

Timerless Cache Side Channels. In environments without a timer, a standard approach for measuring timing differences is to create a timer using counting threads [39, 54, 56]. We compare this approach to PhotoFinish in Section 4.3. The idea of using race conditions with thread-level parallelism for revealing memory access latencies, and thus the foundational principle of PhotoFinish, was proposed by Purnal et al. under the name *architectural reordering* [49]. Our work extensively evaluates and extends this idea. Other fully timerless cache side channels typically rely on architecture-specific properties. For instance, the abort semantics of Intel TSX may reveal cache state information [13, 25, 33]. Similarly, weak coherence between the data and instruction caches on many RISC architectures is known to enable timerless attack primitives [21, 31, 62, 63]. Zhang et al. show that the `umonitor` and `umwait` instructions on x86-based architectures provide unprivileged users with architectural feedback on cache activity [75]. Yu et al. create S^2C, a timerless cache side channel for Apple's M1 architecture based on hardware synchronization instructions [73]. Thomas et al. propose ExfilState, a fuzzing-based technique for discovering architectural cache side channels on ARM [63]. In a follow-up work, Thomas et al. [62] show that cache coherency effects in RISC CPUs can reveal information on the instruction cache's state.

Attacks with Imprecise Timers. In addition to fully timerless attacks like PhotoFinish, a large body of work exists on cache attacks with imprecise timers. This line of work is orthogonal to PhotoFinish, as the limiting factor in cache attacks is often the resolution of a timer, not its full absence. Many rely on *amplification*, i.e., magnifying the timing differences between the cached and uncached states until they are measurable even with a coarse-grained timer. A seminal work is `leaky.page` [61], which introduces an amplification primitive that relies on the Tree-PLRU cache replacement policy used in the L1 caches of many microarchitectures. Purnal et al. extend this primitive, significantly increasing its amplification ratio [49], and propose another amplification primitive using non-temporal prefetch instructions on x86. Katzman et al. propose an amplification method using logic gates operating on the cache state [32]. It propagates the cache state of a target address in a tree-like structure, creating an amplified timing difference when accessing the leaves sequentially. Morgan et al. introduce comparator gates, which create a race condition between two memory accesses during speculative execution, encoding the result in the cache [43]. This amplifies a timing difference on the order of individual cycles to a difference between L1 and DRAM. Xiao et al. propose Hacky Racers [71], a primitive relying on instruction-level parallelism introduced by out-of-order execution. It encodes the relative completion order of two parallelizable instruction paths in the cache and uses amplification to make it accessible with a coarse-grained timer. An orthogonal approach to amplification is to average many direct measurements with a coarse-grained timer. This requires many samples per target and was demonstrated mainly in remote attacks [10, 55, 57].

6.3 Mitigations

Eliminating Shared Memory. Shared memory, a requirement for PhotoFinish, can often be replaced with message passing via channels. This alone does not fully mitigate PhotoFinish, as it could, in theory, also be implemented with channels by replacing writes to s and w with channel writes. However, channels are often managed components, e.g., by the operating system, and thus provide a surface for implementing mitigations. In many web browsers, for instance, message passing is already rate-limited to prevent the construction of counting threads [35, 54]. To further mitigate PhotoFinish, channels can add a random delay before delivering a message. This makes the race-condition outcome random, preventing attackers from inferring usable information from a single measurement. Another mitigation is to grant write access to a shared memory segment to only one thread at a time. PhotoFinish requires w to be writable by both threads, which such a configuration prevents.

Eliminating True Parallelism. Many programs follow an asynchronous programming model, forgoing true parallelism in favor of weaker, interleaving-based concurrency. While this does not bring the full performance benefits of parallelism, it can unblock expensive operations, allowing programs to continue executing while waiting (e.g., for the operating system). A prominent example is asynchronous functions in languages such as JavaScript. Implementing PhotoFinish is not possible in such an environment, as it requires parallelism.

7 Conclusion

We presented PhotoFinish, a timerless probing primitive for distinguishing microarchitectural latencies. In contrast to other timerless primitives, PhotoFinish is portable across microarchitectures and relies only on thread-level parallelism and shared memory. We performed an extensive evaluation of PhotoFinish on 38 CPUs, demonstrating that it can reliably separate microarchitectural latency differences on all of them. Additionally, we showed that PhotoFinish works in a web browser, with a temporal accuracy approaching that of our native implementations. Our case studies showed that PhotoFinish applies in a wide variety of contexts and can, in some instances, distinguish latency differences on the order of individual cycles. Overall, PhotoFinish shows that timing-based microarchitectural attacks, even with fine-grained latency differences, do not inherently require a timer.

References

[1] Ieee standard for floating-point arithmetic. *IEEE Std 754-2008*, 2008.

[2] AMD64 Architecture Programmer's Manual, 2024.

[3] Andreas Abel and Jan Reineke. uops.info: Characterizing latency, throughput, and port usage of instructions on intel microarchitectures. In *ASPLOS*, 2019.

[4] Onur Acıiçmez, Çetin Kaya Koç, and Jean-Pierre Seifert. Predicting secret keys via branch prediction. In *CT-RSA 2007*, 2006.

[5] Alejandro Cabrera Aldaya, Billy Bob Brumley, Sohaib ul Hassan, Cesar Pereida García, and Nicola Tuveri. Port contention for fun and profit. In *S&P*, 2019.

[6] Thomas Allan, Billy Bob Brumley, Katrina Falkner, Joop Van de Pol, and Yuval Yarom. Amplifying Side Channels Through Performance Degradation. In *ACSAC*, 2016.

[7] Marc Andrysco, David Kohlbrenner, Keaton Mowery, Ranjit Jhala, Sorin Lerner, and Hovav Shacham. On subnormal floating point and abnormal timing. In *S&P*, 2015.

[8] ARM. Learn the architecture: Generic timer. [Online]. Available: https://developer.arm.com/documentation/102379/0000/The-processor-timers, January 2021.

[9] Alexis Bagia, Vincent Quentin Ulitzsch, Daniël Trujillo, Mengyuan Li, Mengjia Yan, and Jean-Pierre Seifert. A close look at rmp entry caching and its security implications in sev-snp. In *HASP*, 2025.

[10] Daniel J. Bernstein. Cache-Timing Attacks on AES, 2005.

[11] Samira Briongos, Pedro Malagón, José M Moya, and Thomas Eisenbarth. RELOAD+REFRESH: Abusing Cache Replacement Policies to Perform Stealthy Cache Attacks. In *USENIX Security*, 2020.

[12] Claudio Canella, Jo Van Bulck, Michael Schwarz, Moritz Lipp, Benjamin von Berg, Philipp Ortner, Frank Piessens, Dmitry Evtyushkin, and Daniel Gruss. A Systematic Evaluation of Transient Execution Attacks and Defenses. In *USENIX Security*, 2019.

[13] Craig Disselkoen, David Kohlbrenner, Leo Porter, and Dean Tullsen. Prime+Abort: A Timer-Free High-Precision L3 Cache Attack using Intel TSX. In *USENIX Security Symposium*, 2017.

[14] Dmitry Evtyushkin, Ryan Riley, Nael Abu-Ghazaleh, and Dmitry Ponomarev. BranchScope: A New Side-Channel Attack on Directional Branch Predictor. In *ASPLOS*, 2018.

[15] Lukas Gerlach, Simon Schwarz, Nicolas Faroß, and Michael Schwarz. Efficient and generic microarchitectural hash-function recovery. In *S&P*, 2024.

[16] Lukas Gerlach, Daniel Weber, Ruiyi Zhang, and Michael Schwarz. A Security RISC: Microarchitectural Attacks on Hardware RISC-V CPUs. In *S&P*, 2023.

[17] Ben Gras, Kaveh Razavi, Herbert Bos, and Cristiano Giuffrida. Translation Leak-aside Buffer: Defeating Cache Side-channel Protections with TLB Attacks. In *USENIX Security*, 2018.

[18] Ben Gras, Kaveh Razavi, Erik Bosman, Herbert Bos, and Cristiano Giuffrida. ASLR on the Line: Practical Cache Attacks on the MMU. In *NDSS*, 2017.

[19] Daniel Gruss, Clémentine Maurice, Anders Fogh, Moritz Lipp, and Stefan Mangard. Prefetch Side-Channel Attacks: Bypassing SMAP and Kernel ASLR. In *CCS*, 2016.

[20] Daniel Gruss, Clémentine Maurice, Klaus Wagner, and Stefan Mangard. Flush+Flush: A Fast and Stealthy Cache Attack. In *DIMVA*, 2016.

[21] Roberto Guanciale, Hamed Nemati, Christoph Baumann, and Mads Dam. Cache storage channels: Alias-driven attacks and verified countermeasures. In *S&P*, 2016.

[22] Jawad Haj-Yahya, Lois Orosa, Jeremie S Kim, Juan Gómez Luna, A Giray Yağlıkçı, Mohammed Alser, Ivan Puddu, and Onur Mutlu. IChannels: Exploiting current management mechanisms to create covert channels in modern processors. In *ISCA*, 2021.

[23] Lorenz Hetterich and Michael Schwarz. Branch Different - Spectre Attacks on Apple Silicon. In *DIMVA*, 2022.

[24] Tristan Hornetz and Michael Schwarz. PortPrint: Identifying Inaccessible Code with Port Contention. In *uASC*, 2025.

[25] Tristan Hornetz, Hosein Yavarzadeh, Albert Cheu, Adria Gascon, Lukas Gerlach, Daniel Moghimi, Phillipp Schoppmann, Michael Schwarz, and Ruiyi Zhang. TDXRay: Microarchitectural Side-Channel Analysis of Intel TDX for Real-World Workloads. In *IEEE S&P*, 2026.

[26] Herbert HJ Hum and James R Goodman. Forward state for use in cache coherency in a multiprocessor system, 2005. US Patent 6,922,756.

[27] Intel. Intel Software Guard Extensions (Intel SGX) Developer Guide, October 2021. Rev 2.15.1.

[28] Intel. Intel 64 and IA-32 architectures software developer's manual combined volumes: 1, 2A, 2B, 2C, 2D, 3A, 3B, 3C, 3D and 4, 2024.

[29] Intel Corporation. Intel 64 and IA-32 Architectures Optimization Reference Manual Volume 1, 2023.

[30] Gorka Irazoqui, Thomas Eisenbarth, and Berk Sunar. Systematic reverse engineering of cache slice selection in intel processors. In *DSD*, 2015.

[31] Yu Jin, Minghong Sun, Dongsheng Wang, Pengfei Qiu, Yinqian Zhang, and Shuwen Deng. Ghostcache: Timer- and counter-free cache attacks exploiting weak coherence on risc-v and arm chips. In *CCS*, 2025.

[32] Daniel Katzman, William Kosasih, Chitchanok Chuengsatiansup, Eyal Ronen, and Yuval Yarom. The gates of time: Improving cache attacks with transient execution. In *USENIX Security*, 2023.

[33] Sowoong Kim, Myeonggyun Han, and Woongki Baek. Dprime+dabort: A high-precision and timer-free directory-based side-channel attack in non-inclusive cache hierarchies using intel tsx. In *HPCA*, pages 67–81, 2022.

[34] Paul Kocher, Jann Horn, Anders Fogh, Daniel Genkin, Daniel Gruss, Werner Haas, Mike Hamburg, Moritz Lipp, Stefan Mangard, Thomas Prescher, Michael Schwarz, and Yuval Yarom. Spectre Attacks: Exploiting Speculative Execution. In *S&P*, 2019.

[35] David Kohlbrenner and Hovav Shacham. Trusted browsers for uncertain times. In *USENIX Security Symposium*, 2016.

[36] David Kohlbrenner and Hovav Shacham. On the effectiveness of mitigations against floating-point timing channels. In *USENIX Security*, 2017.

[37] Sangho Lee, Ming-Wei Shih, Prasun Gera, Taesoo Kim, Hyesoon Kim, and Marcus Peinado. Inferring Fine-grained Control Flow Inside SGX Enclaves with Branch Shadowing. In *USENIX Security Symposium*, 2017.

[38] Moritz Lipp, Daniel Gruss, and Michael Schwarz. AMD Prefetch Attacks through Power and Time. In *USENIX Security*, 2022.

[39] Moritz Lipp, Daniel Gruss, Raphael Spreitzer, Clémentine Maurice, and Stefan Mangard. ARMageddon: Cache Attacks on Mobile Devices. In *USENIX Security*, 2016.

[40] Moritz Lipp, Vedad Hadžić, Michael Schwarz, Arthur Perais, Clémentine Maurice, and Daniel Gruss. Take a Way: Exploring the Security Implications of AMD's Cache Way Predictors. In *AsiaCCS*, 2020.

[41] Moritz Lipp, Michael Schwarz, Daniel Gruss, Thomas Prescher, Werner Haas, Anders Fogh, Jann Horn, Stefan Mangard, Paul Kocher, Daniel Genkin, Yuval Yarom, and Mike Hamburg. Meltdown: Reading Kernel Memory from User Space. In *USENIX Security*, 2018.

[42] Fangfei Liu, Yuval Yarom, Qian Ge, Gernot Heiser, and Ruby B. Lee. Last-Level Cache Side-Channel Attacks are Practical. In *S&P*, 2015.

[43] Bradley Morgan, Gal Horowitz, Sioli O'Connell, Stephan van Schaik, Chitchanok Chuengsatiansup, Daniel Genkin, Olaf Maennel, Paul Montague, Eyal Ronen, and Yuval Yarom. Slice+slice baby: Generating last-level cache eviction sets in the blink of an eye. In *S&P*, 2025.

[44] Yossef Oren, Vasileios P Kemerlis, Simha Sethumadhavan, and Angelos D Keromytis. The Spy in the Sandbox: Practical Cache Attacks in JavaScript and their Implications. In *CCS*, 2015.

[45] Dag Arne Osvik, Adi Shamir, and Eran Tromer. Cache Attacks and Countermeasures: the Case of AES. In *CT-RSA*, 2006.

[46] Riccardo Paccagnella, Licheng Luo, and Christopher W Fletcher. Lord of the ring(s): Side channel attacks on the CPU on-chip ring interconnect are practical. In *USENIX Security*, 2021.

[47] Mark S Papamarcos and Janak H Patel. A low-overhead coherence solution for multiprocessors with private cache memories. In *ISCA*, 1984.

[48] Peter Pessl, Daniel Gruss, Clémentine Maurice, Michael Schwarz, and Stefan Mangard. DRAMA: Exploiting DRAM Addressing for Cross-CPU Attacks. In *USENIX Security Symposium*, 2016.

[49] Antoon Purnal, Marton Bognar, Frank Piessens, and Ingrid Verbauwhede. Showtime: Amplifying arbitrary cpu timing side channels. In *AsiaCCS*, 2023.

[50] Antoon Purnal, Furkan Turan, and Ingrid Verbauwhede. Prime+Scope: Overcoming the Observer Effect for High-Precision Cache Contention Attacks. In *CCS*, 2021.

[51] Mikka Rainer, Lorenz Hetterich, Fabian Thomas, Tristan Hornetz, Leon Trampert, Lukas Gerlach, and Michael Schwarz. Rapid reversing of non-linear CPU cache slice functions: Unlocking physical address leakage. In *S&P*, 2025.

[52] Fabian Rauscher, Carina Fiedler, Andreas Kogler, and Daniel Gruss. A systematic evaluation of novel and existing cache side channels. In *NDSS*, 2025.

[53] Thomas Rokicki, Clémentine Maurice, and Michael Schwarz. CPU Port Contention Without SMT. In *ESORICS*, 2022.

[54] Michael Schwarz, Clémentine Maurice, Daniel Gruss, and Stefan Mangard. Fantastic timers and where to find them: High-resolution microarchitectural attacks in JavaScript. In *FC*, 2017.

[55] Michael Schwarz, Martin Schwarzl, Moritz Lipp, Jon Masters, and Daniel Gruss. Netspectre: Read arbitrary memory over network. In *ESORICS*, 2019.

[56] Michael Schwarz, Samuel Weiser, Daniel Gruss, Clémentine Maurice, and Stefan Mangard. Malware guard extension: abusing intel sgx to conceal cache attacks. *Cybersecurity*, 3(1):2, 2020.

[57] Martin Schwarzl, Pietro Borrello, Andreas Kogler, Kenton Varda, Thomas Schuster, Michael Schwarz, and Daniel Gruss. Robust and scalable process isolation against spectre in the cloud. In *ESORICS*, 2022.

[58] Anatoly Shusterman, Ayush Agarwal, Sioli O'Connell, Daniel Genkin, Yossi Oren, and Yuval Yarom. Prime+Probe 1, JavaScript 0: Overcoming Browser-based Side-Channel Defenses. In *USENIX Security Symposium*, 2021.

[59] SiFive, Inc. SiFive Inclusive Cache (block-inclusivecache-sifive), 2019. Online: https://github.com/sifive/block-inclusivecache-sifive - Accessed 2026-08-01.

[60] Daniel J. Sorin, Mark D. Hill, and David A. Wood. *A Primer on Memory Consistency and Cache Coherence*. 2011.

[61] Stephen Röttger and Artur Janc. A Spectre proof-of-concept for a Spectre-proof web, 2021.

[62] Fabian Thomas and Michael Schwarz. RISCy Cache Coherence: Timer-Free Architectural Cache Attacks via Instruction/Data Cache Incoherence. In *IEEE S&P*, 2026.

[63] Fabian Thomas, Michael Torres, Daniel Moghimi, and Michael Schwarz. ExfilState: Automated Discovery of Timer-Free Cache Side Channels on ARM CPUs. In *CCS*, 2025.

[64] Tom Van Goethem, Wouter Joosen, and Nick Nikiforakis. The clock is still ticking: Timing attacks in the modern web. In *CCS*, 2015.

[65] Anne van Kesteren. Safely reviving shared memory, 2020.

[66] Stephan Van Schaik, Cristiano Giuffrida, Herbert Bos, and Kaveh Razavi. Malicious Management Unit: Why Stopping Cache Attacks in Software is Harder Than You Think. In *USENIX Security Symposium*, 2018.

[67] Pepe Vila, Boris Köpf, and Jose Morales. Theory and practice of finding eviction sets. In *S&P*, 2019.

[68] W3C. Web Workers - W3C Working Draft 24 September 2015, 2015.

[69] Daniel Weber, Leonard Niemann, Lukas Gerlach, Jan Reineke, and Michael Schwarz. No leakage without state change: Repurposing configurable cpu exceptions to prevent microarchitectural attacks. In *ACSAC*, 2024.

[70] John C Wray. An Analysis of Covert Timing Channels. *Journal of Computer Security*, 1992.

[71] Haocheng Xiao and Sam Ainsworth. Hacky racers: Exploiting instruction-level parallelism to generate stealthy fine-grained timers. In *ASPLOS*, 2023.

[72] Yuval Yarom and Katrina Falkner. Flush+Reload: A high resolution, low noise, L3 cache side-channel attack. In *USENIX Security*, 2014.

[73] Jiyong Yu, Aishani Dutta, Trent Jaeger, David Kohlbrenner, and Christopher W Fletcher. Synchronization Storage Channels (S^2C): Timer-less Cache Side-Channel Attacks on the Apple M1 via Hardware Synchronization Instructions. In *USENIX Security*, 2023.

[74] Ruiyi Zhang, Tristan Hornetz, Lukas Gerlach, and Michael Schwarz. Taming the Linux Memory Allocator for Rapid Prototyping. In *DIMVA*, 2025.

[75] Ruiyi Zhang, Taehyun Kim, Daniel Weber, and Michael Schwarz. (M)WAIT for It: Bridging the Gap between Microarchitectural and Architectural Side Channels. In *USENIX Security*, 2023.

A Per-Machine Experimental Results

We present the per-machine experimental results from Section 4.1 and Section 4.2 in Table 2 and Table 1. F_{slow}^{fast} denotes the F_1 score of the unpadded PhotoFinish in a specific setting, and G_{slow}^{fast} denotes the width of the golden range with padded PhotoFinish in `add`-cycles. CTR is the average edge width of a counting thread, and Δ_{DRAM}^{L1} is the latency difference between an L1 hit and a DRAM access as measured in Section 4.1. Table 3 lists the results of our WASM-based implementation on x86_64.

B Reliable Cache Eviction

Many experiments in this paper rely on eviction sets to prime the cache state, e.g., to evict an address from the L2 or L3 cache. Unless otherwise specified, as in Section 5.2, we construct these eviction sets using the physical addresses of both the target and eviction-set addresses. Our eviction sets consist of $n+2$ set-index-congruent addresses for evicting the L1 cache, and $n+4$ addresses for evicting the L2 cache, where n is the associativity of the targeted cache level. The overfill provides additional robustness. On CPUs where the L1 cache has higher associativity than the L2, such as Intel's Skylake architectures, we use the larger L1 associativity for n when evicting the L2 cache. To apply an eviction set, we first access the target address to cache it in the L1 cache. Then, we access all eviction-set addresses in a linear pass, then in multiple passes in random order, and then again in a reverse-order linear pass. To our knowledge, all L1 caches in our evaluation are directly indexed, i.e., there is no complex hash function for determining the cache set index. Testing for congruency is therefore trivial. With the L2 cache, we assume direct indexing for all CPUs except for Apple's M1 and M2 architectures and AMD's Zen 3, 4, and 5 architectures. For M1, we rely on the set index function reverse-engineered by Yu et al. [73], and for M2, we find L2 eviction sets using the algorithm described by Vila et al. [67]. For AMD, we use the set index functions reverse-engineered by Bagia et al. [9] and Zhang et al. [74] to determine cache set congruency. Where possible, we rely on cache management instructions to flush addresses from the LLC. On architectures where such instructions are privileged, we use a simple kernel module to grant user-mode processes access. We still perform LLC eviction with eviction sets only on the StarFive JH7100 and the Microchip MPFS025T. Based on publicly available RTL code [59], we assume direct set indexing and construct the LLC eviction sets in the same way as L2 eviction sets on other platforms.

Table 1: PhotoFinish characteristics with `T0` on core 0 and `T1` on the SMT sibling core. We list Δ_{sync}, Δ^{L1}_{DRAM}, and the average edge width of a counting thread (CTR). Additionally, it shows the F_1 score of unpadded race probing (F^{fast}_{slow}) and the golden range's length for padded race probing (G^{fast}_{slow}). All values, except the F_1 scores, are measured in `add`-cycles.

	CPU	μArchitecture				Unpadded		Padded			
			Δ_{sync}	Δ^{L1}_{DRAM}	CTR	F^{L1}_{DRAM}	F^{LLC}_{DRAM}	G^{L1}_{DRAM}	G^{L1}_{L2}	G^{L1}_{L3}	G^{LLC}_{DRAM}
x86	AMD EPYC 7252	Zen 2 (Rome)	43 ± 1	329	74	1.00	0.99	41	5	31	12
	AMD Ryzen 7 5700G	Zen 3 (Cezanne)	55 ± 22	322	90	0.99	0.77	6	·	·	·
	AMD Ryzen 9 7950X3D	Zen 4 (Raphael)	38 ± 29	496	95	0.99	0.61	5	·	·	·
	AMD EPYC 9124	Zen 4 (Genoa)	50 ± 19	324	96	0.99	0.23	9	·	2	·
	AMD Ryzen 9 9950X	Zen 5 (Granite Ridge)	76 ± 31	462	249	0.99	0.93	37	·	·	·
	Intel Core i3-2120	Sandy Bridge	47 ± 8	200	50	1.00	1.00	36	·	·	43
	Intel Core i3-5010U	Broadwell	76 ± 7	156	43	0.99	0.99	18	·	·	14
	Intel Xeon E3-1505M v5	Skylake	35 ± 13	170	42	1.00	0.81	20	·	·	·
	Intel Pentium Gold G5400T	Coffee Lake	44 ± 8	205	41	1.00	0.94	39	·	25	·
	Intel Core i7-8700	Coffee Lake	35 ± 18	272	25	1.00	0.24	39	·	39	·
	Intel Core i7-10710U	Comet Lake	35 ± 15	305	43	1.00	0.92	20	·	·	·
	Intel Core i7-1185G7	Willow Cove	54 ± 21	513	41	0.99	0.88	32	·	8	·
	Intel Xeon Gold 6346	Sunny Cove	47 ± 25	233	47	1.00	1.00	27	·	26	31
	Intel Core i7-11700	Cypress Cove	53 ± 20	316	46	0.99	0.98	28	·	18	6
	Intel Core i9-12900H	Golden Cove (P)	68 ± 22	345	49	1.00	0.99	40	·	14	53
	Intel Core i5-13420H	Raptor Cove (P)	71 ± 18	345	56	1.00	0.99	45	·	18	50
	Intel Core i9-13900K	Raptor Cove (P)	70 ± 15	456	58	0.99	0.94	43	·	25	·
	Intel Xeon Gold 6526Y	Raptor Cove	54 ± 30	357	58	1.00	0.10	42	·	35	·
	Intel Xeon 6736P	Redwood Cove	60 ± 17	681	50	1.00	0.94	45	·	39	·
LA64	Loongson 3A6000	3A6000	22 ± 8	183	3	1.00	0.00	14	·	14	·

Table 2: PhotoFinish characteristics with `T0` on core 0 and `T1` on the physical core with the next-highest ID. We list Δ_{sync}, Δ^{L1}_{DRAM}, and the average edge width of a counting thread (CTR). Additionally, it shows the F_1 score of unpadded race probing (F^{fast}_{slow}) and the golden range's length for padded race probing (G^{fast}_{slow}). All values, except the F_1 scores, are measured in `add`-cycles.

	CPU	μArchitecture				Unpadded		Padded			
			Δ_{sync}	Δ^{L1}_{DRAM}	CTR	F^{L1}_{DRAM}	F^{LLC}_{DRAM}	G^{L1}_{DRAM}	G^{L1}_{L2}	G^{L1}_{L3}	G^{LLC}_{DRAM}
ARM	Apple M1	Icestorm (E)	122 ± 1	236	86	0.99	0.99	177	8	88	87
		Firestorm (P)	142 ± 1	342	105	1.00	0.99	230	53	122	44
	Apple M2	Blizzard (E)	123 ± 6	290	97	0.97	0.99	123	·	24	65
		Avalanche (P)	142 ± 7	371	112	0.97	0.99	92	·	·	355
	Broadcom BCM2837	Cortex-A53	49 ± 0	161	35	0.99	0.99	50	13	–	37
	Broadcom BCM2712	Cortex-A76	143 ± 61	322	172	0.99	0.99	161	·	·	234
	Huawei Kunpeng 920	TaiShan-v110	243 ± 1	271	91	0.75	0.75	104	·	·	92
	Ampere Altra Q64-30	Altra	293 ± 20	251	242	0.67	0.67	99	·	·	120
x86	AMD EPYC 7252	Zen 2 (Rome)	78 ± 0	329	205	0.99	0.99	78	10	34	44
	AMD Ryzen 7 5700G	Zen 3 (Cezanne)	84 ± 0	322	162	0.99	0.99	71	·	33	37
	AMD Ryzen 9 7950X3D	Zen 4 (Raphael)	87 ± 3	496	183	1.00	0.99	66	·	6	38
	AMD EPYC 9124	Zen 4 (Genoa)	85 ± 5	324	195	0.99	0.84	49	·	·	·
	AMD Ryzen 9 9950X	Zen 5 (Granite Ridge)	140 ± 1	462	201	0.99	0.99	125	10	47	78
	Intel Core i3-2120	Sandy Bridge	71 ± 8	200	102	0.99	0.99	61	2	7	62
	Intel Core i5-4690	Haswell	95 ± 1	281	116	0.99	1.00	89	·	13	100
	Intel Core i3-5010U	Broadwell	88 ± 3	156	117	0.99	0.99	88	2	12	54
	Intel Xeon E3-1505M v5	Skylake	90 ± 1	170	126	1.00	1.00	97	2	21	68
	Intel Core i5-6400T	Skylake	95 ± 5	88	134	0.68	0.68	80	·	19	50
	Intel Pentium G4400T	Skylake	88 ± 4	190	120	0.99	0.99	92	·	·	63
	Intel Pentium Gold G5400T	Coffee Lake	92 ± 3	205	146	1.00	1.00	91	·	22	59
	Intel Core i7-8700	Coffee Lake	96 ± 3	272	114	1.00	0.99	95	·	27	54
	Intel Core i7-9700K	Coffee Lake	91 ± 5	298	156	1.00	0.99	103	·	22	73
	Intel Core i7-10710U	Comet Lake	95 ± 3	305	224	0.99	0.99	90	·	21	57
	Intel Core i7-1185G7	Willow Cove	126 ± 17	513	231	0.99	0.99	103	·	9	55
	Intel Xeon Gold 6346	Sunny Cove	151 ± 6	233	300	1.00	0.69	169	12	50	143
	Intel Core i7-11700	Cypress Cove	134 ± 12	316	241	0.99	1.00	131	·	·	83
	Intel Core i9-12900H	Gracemont (E)	147 ± 3	295	194	1.00	1.00	147	14	68	74
		Golden Cove (P)	145 ± 22	345	244	0.99	0.99	133	·	24	82
	Intel Core i5-13420H	Gracemont (E)	156 ± 4	288	198	0.99	0.99	157	7	33	113
		Raptor Cove (P)	142 ± 15	345	258	0.99	0.99	92	·	30	21
	Intel Core i9-13900K	Gracemont (E)	170 ± 10	392	166	1.00	1.00	167	14	52	97
		Raptor Cove (P)	179 ± 13	456	357	0.99	0.99	156	·	31	61
	Intel Xeon Gold 6526Y	Raptor Cove	143 ± 8	357	355	0.99	0.99	169	8	65	93
	Intel Core Ultra 9 288V	Lion Cove	153 ± 13	279	286	0.99	0.99	135	·	38	71
	Intel Core Ultra 9 285K	Skymont (E)	98 ± 0	475	86	1.00	1.00	97	·	10	33
		Lion Cove (P)	162 ± 17	577	266	0.99	0.99	234	·	40	172
	Intel Xeon 6736P	Redwood Cove	403 ± 23	681	792	0.99	0.99	396	·	138	193
	Intel Core Ultra 5 325	Darkmont (E)	90 ± 7	121	86	0.94	0.93	63	·	·	47
		Cougar Cove (P)	151 ± 25	284	202	1.00	0.99	150	·	28	88
RISC-V	StarFive JH7100	U74	27 ± 15	129	55	0.98	0.59	2	·	–	·
	Microchip MPFS025T	U54	18 ± 0	75	54	0.99	0.66	19	·	–	·
	T-Head TH1520	C910	56 ± 0	239	86	0.99	0.99	48	29	–	48
	SpacemiT K1	X60	75 ± 7	145	216	0.99	1.00	55	2	–	45
LA64	Loongson 3A5000	3A5000	186 ± 2	309	123	0.99	1.00	150	·	5	140
	Loongson 3A6000	3A6000	120 ± 2	183	100	1.00	1.00	119	·	33	84

Table 3: PhotoFinish performance in WebAssembly (Chromium v147.0) in the L1 vs. DRAM setting on x86_64. Δ_{sync} and the width of the golden range G^{L1}_{DRAM} are measured in `add`-cycles.

CPU	**μArchitecture**	Δ_{sync}		F^{L1}_{DRAM} (unpadded)		G^{L1}_{DRAM} (padded)	
		SMT	Cross-core	SMT	Cross-core	SMT	Cross-core
AMD EPYC 7252	Zen 2 (Rome)	50 ± 4	101 ± 33	0.99	0.99	38	67
AMD Ryzen 7 5700G	Zen 3 (Cezanne)	48 ± 20	126 ± 3	0.99	0.99	44	113
AMD Ryzen 9 7950X3D	Zen 4 (Raphael)	49 ± 21	88 ± 13	0.99	0.99	50	69
AMD EPYC 9124	Zen 4 (Genoa)	48 ± 18	120 ± 1	0.99	0.99	42	128
AMD Ryzen 9 9950X	Zen 5 (Granite Ridge)	91 ± 7	165 ± 7	0.99	0.99	69	137
Intel Core i3-2120	Sandy Bridge	24 ± 12	84 ± 30	0.99	0.98	22	36
Intel Core i5-4690	Haswell	–	83 ± 1	–	0.99	–	69
Intel Core i3-5010U	Broadwell	28 ± 14	108 ± 4	0.99	0.99	25	61
Intel Xeon E3-1505M v5	Skylake	33 ± 20	89 ± 7	0.99	0.99	41	66
Intel Core i5-6400T	Skylake	–	99 ± 5	–	0.97	–	50
Intel Pentium G4400T	Skylake	–	91 ± 43	–	0.99	–	26
Intel Pentium Gold G5400T	Coffee Lake	36 ± 17	128 ± 6	0.99	0.99	30	98
Intel Core i7-8700	Coffee Lake	36 ± 17	110 ± 26	0.99	0.99	28	75
Intel Core i7-9700K	Coffee Lake	–	106 ± 4	–	0.98	–	82
Intel Core i7-10710U	Comet Lake	32 ± 19	120 ± 12	0.99	0.99	11	29
Intel Core i7-1185G7	Willow Cove	46 ± 11	122 ± 18	0.99	0.99	41	90
Intel Xeon Gold 6346	Sunny Cove	36 ± 17	327 ± 113	0.99	0.99	37	316
Intel Core i7-11700	Cypress Cove	39 ± 6	130 ± 5	0.99	0.99	11	88
Intel Core i9-12900H	Gracemont (E)	–	140 ± 5	–	0.99	–	125
	Golden Cove (P)	55 ± 16	229 ± 37	0.99	0.99	27	139
Intel Core i5-13420H	Gracemont (E)	–	128 ± 6	–	0.99	–	73
	Raptor Cove (P)	55 ± 16	193 ± 17	0.99	0.99	17	155
Intel Core i9-13900K	Gracemont (E)	–	168 ± 4	–	0.99	–	123
	Raptor Cove (P)	54 ± 15	276 ± 16	0.99	0.99	31	188
Intel Xeon Gold 6526Y	Raptor Cove	49 ± 28	303 ± 21	0.99	0.99	33	191
Intel Core Ultra 9 288V	Lion Cove	–	173 ± 23	–	0.99	–	152
Intel Core Ultra 9 285K	Skymont (E)	–	180 ± 2	–	0.97	–	142
	Lion Cove (P)	–	326 ± 31	–	0.99	–	218
Intel Xeon 6736P	Redwood Cove	53 ± 20	974 ± 31	0.99	0.99	35	361
Intel Core Ultra 5 325	Darkmont (E)	–	130 ± 100	–	0.99	–	14
	Cougar Cove (P)	–	172 ± 12	–	0.99	–	128